

Feature: Reschedule

From build-plan: feature 7b

Branch: feature/07b-reschedule

Status: verified. Whole feature seen and agreed by Frank 2026-10-03; steps 7b.1 to 7b.5 built, tested and reviewed step by step; no P0 or P1 was ever open; decisions 10 to 14 made with Frank along the way. The checkpoint for the final review.

Goal

On the page Jane's private link opens (feature 7a), she can move her booking to another time. She sees the free times for the same service, and the booking's own old time never stands in the way: neither the time it holds nor its own event in the booked person's Google. She picks one, confirms, and the same booking moves: its held time, its Google event and her calendar invite move with it, she and the business are told by email, and her timeline shows it. Nobody signs in: the link is still the key. This finishes feature 7.

In scope

- Free times for moving one booking: the same service, read as for a new booking, with the booking's own held time and its own Google event left out.
- Move: one public route, one transaction (the old held time released, the new one held, the booking's times changed, a `booking_moved` timeline entry), safe to press twice.
- After the move, without Jane's answer waiting: the booked person's Google event updated in place, an email to Jane with her invite updated, and one to the business.
- Jane's page: "Change the time", the free times a week at a time, a confirm step, and every state (no free times that week, a time taken meanwhile, already started, cancelled, failure).
- The slug `bookings` reserved, so no business can sit where these routes do.

Decisions made in the spec

1. **The same booking moves; nothing is cancelled and rebooked.** Its id, its link, its lead and its timeline stay; only its times (and its room, when a room is needed) change. A cancelled-and-rebooked booking would send Jane a cancellation she did not ask for and break her link. 2. **The old time never stands in the way.** The free times for a move leave out the booking's own held rows (person and room) and its own Google event, so moving 30 minutes later is offered even when the new time overlaps the old one. Everyone else's time counts as usual: bookable hours, notice, horizon, other bookings, time off, Google. 3. **A move is checked again when it lands.** The chosen start must still be one of the free times for this

booking at that moment; the database's overlap rule is the last word (5a). Taken meanwhile: 409 `time_taken`, nothing changes, the page offers the times again. 4. **Moving twice to the same time is one move.** A press whose start equals the booking's current start answers the booking as it is: no new timeline entry, no Google call, no email. 5. **A move never waits for Google or Resend**, as the booking and the cancel (feature 6's decision 1, 7a's decision 5). A failed update or email logs one line; feature 8 retries. 6. **Every change carries a higher invite number.** The booking gains `sequence` (0 when made). Each move adds one, and its invite carries it; a cancel's invite carries `sequence + 1`. Calendars ignore an update whose number is not higher than the last one they saw (7a.4's note). 7. **Google's event is updated in place, never deleted and written again** (7a.3's review): Google keeps a deleted event's id and refuses a new event with it. An event that was never written (404) is written then. 8. **Both are told.** Jane: "Your booking has moved", the new time and the old one, her invite updated. The business: "Moved: <service>, <new time>", with the old time and who moved it, Reply to the customer. One email each per move (keys carry the move's number). 9. **The page still names the business, never the product** (7a's decision 9), and never shows the customer's details (7a's decision 3). 10. **Changing the time is booking again** (Frank, 2026-10-03, open question 1). Jane gets the same choices as her first booking form, for the same service: any available, or a named person who offers it, and only the times that are really free, with the person who has the skill, a free room and the time matching again. Her booking stays hers while she looks; the moment she confirms, the new time is held and the old one released together, and a time taken meanwhile leaves her booking as it was. Rejected: only her current person's times, which no new customer is limited to either. 11. **Her own Google event is crossed off Google's busy answer** (Frank, 2026-10-03, open question 2). Free times are still read from Google's free/busy, as for every booking; for her current person, the booking's own start to end is taken out of the busy blocks Google answers. One rare slip is accepted: another event the person put on top of her booking is hidden for that overlap. Rejected: reading the person's events one by one and skipping hers, exact but rebuilding Google's own rules (all-day events, events marked free, declined invitations), with more to get wrong on every move. 12. **Jane may move until the appointment starts** (Frank, 2026-10-03, open question 3), the same rule as cancel (7a's decision 11). From its start the page shows the business's phone instead. A business's own cutoff comes with Settings, feature 12. Rejected: a fixed 24-hour cutoff for every business, a rule nobody chose. 13. **Switching a service off stops only new bookings** (Frank, 2026-10-03, from 7b.1's review, F-138). A booking already made keeps its page whole: Change the time and Cancel, its move times read for its own service even when the service is switched off. Rejected: hiding the move and showing the phone, which treats an existing booking as if it were new. 14. **Changing the time starts with her own person** (Frank, 2026-10-04, from the final review, F-168). The Who choice opens on the person she is booked with, still changeable to "Any available" or anyone else who offers the service, and the confirm question names who she will be with ("Move to Monday, October 12 at 2:00 p.m. with Mei?", or "with any available person"). If her person no longer offers the service, it opens on "Any available". Rejected: opening on "Any available", which could hand her to another person on a tie without saying so.

Out of scope

- Changing the service, the address or the details: a new booking.

- The owner moving a booking from the dashboard: features 11 and 12b, which can call the same move function.
- A business's own rule for how late a move may happen: Settings, feature 12. Until then it is decision 12: until the start.
- Retrying a failed event update or email: feature 8.
- A text telling the customer: feature 8.
- Rate limits on the public routes: feature 9.

Build loop

Steps are built one at a time on `feature/07b-reschedule`. Each step's plan gets Frank's yes just before it is built. After that yes nothing stops until the review: build, tests, tick the box, the build log entry, commit with the step number and push to the feature branch, `/audit` scoped to the step, then the independent review (`workflow.stepReview: "every"`, `workflow.checkpointCommits: "enabled"`). Findings are talked through after the review; P0/P1 are fixed before the next step. `/complete` makes the merge commit, on Frank's yes.

No package is planned. Installing one is a line only Frank crosses.

Build steps

- ☒ **7b.1 Free times that ignore the booking's own time.**
 - `findFreeTimes` gains `ignoreBooking ( { id, personId, startsAt, endsAt } )`: that booking's held rows are left out of every person's and room's busy time, and its own start to end is crossed off the busy blocks Google answers for its current person only (decision 11, `crossOffSpan`).
 - `GET /public/bookings/:token/times?from=YYYY-MM-DD&to=YYYY-MM-DD&person=<id>` (public CORS, never with credentials, `no-store`): the people she can pick and the free start times for this booking's service, for the picked person or any available when `person` is left out, as the booking form's own times route answers (decision 10), with its query rule (31 dates at most) and its 503 `unavailable` when a calendar cannot be read. 404 for a bad link or a person who does not offer the service (7a's decision 2); 409 `already_started` or `already_cancelled` when it can no longer move.
 - The slug `bookings` is refused when a business is made.
 - **Done when** saved tests, against the local database: a time overlapping the booking's own old time is offered; another person who offers the service is offered too, and any available covers everyone; the same time for another booking's service is not; a room held only by this booking does not block it; another booking, time off and the person's Google still block; the booking's own Google event does not; a bad link answers 404 with the same body; a started or cancelled booking answers 409;

nothing in the answer carries the customer's details; making a business with the slug `bookings` is refused.

- **7b.2 Move.**

- The booking gains `sequence` (integer, 0); `booking_moved` joins the activity types; migration 0018.
- `backend/lib/booking/move-booking.ts` : the start checked against `findFreeTimes` with `ignoreBooking` for the picked person or any available, then one transaction with the booking row locked: still confirmed and not started, its active held rows released, the new span held for the picked person or the first free one in the booking's own any-available order, with the first free room by name, `startsAt`, `endsAt`, `personId`, `placeId` and `sequence + 1` saved, a `booking_moved` entry. The same start with the same person (or any available) as now: answers it, writes nothing (decision 4).
- `POST /public/bookings/:token/move` with `{ startsAt, personId }` (`personId` null for any available): 200 the page's view at the new time; 404 for a bad link; 409 `time_taken`, `already_started` or `already_cancelled`; 400 for a start that is not an ISO instant, or a body that is not JSON; 413 over 1 KB; 503 when a calendar cannot be read.
- The page's view gains `canMove` (the same as `canCancel`, decision 13).
- A booking's own service is read even when switched off, for its move times and its move (decision 13); new bookings still need it switched on.
- **Done when** saved tests, against the local database: a move changes the times, releases the old rows, holds the new ones and writes one entry with the old and new start; moving into a time overlapping the old one works; moving to another person changes the person and holds their time; the free-times route offers the old time again; a second press to the same time writes nothing more; a time taken meanwhile is refused and nothing changes; two moves at the same instant to the same time make one; a started or cancelled booking is refused; another business's booking is never touched; nothing in the answer, the log or the timeline payload carries the customer's details.

- **7b.3 The event moves in the booked person's Google.**

- The provider seam gains `updateEventTime(accessToken, eventId, time)` : Google's `PATCH` of the event's start and end by its saved id; 404 or 410 answers false and the event is written afresh; throws on any other failure. `move-booking-event.ts` does the following.
- When the move changed the person: the event is removed from the old person's Google and written into the new person's. Google keeps a deleted event's id in a calendar, so an event moved to another person carries an id made from the booking and its `sequence`; updates and removals use the saved `calendarEventId`, so moving back to the first person later is never refused (the 7b.3 plan confirms the exact id rule).
- Started by the move after its transaction, not awaited, in the shape of the removals (decision 5); the updates still running can be awaited by tests.

- **Done when** saved tests, with Google faked: a move patches the event with the new times; an event never written gets written; a move to another person removes the old person's event and writes the new person's; moving back to the first person later is not refused; no connection makes no call; the call carries the person's own access key; a Google error keeps the move, logs one line with no customer details; a second press to the same time makes no call; the move's answer does not wait for Google.
- ✓ **7b.4 Both are told.**
 - `bookingIcs` takes `sequence`: the moved invite is `METHOD:REQUEST`, the same UID, the new times and the booking's `sequence`; the cancelling invite carries `sequence + 1` instead of a fixed 1 (decision 6). Stamped with the move's own timeline moment, so a retry sends the same invite.
 - `backend/emails/booking-moved.tsx` to Jane: "Your booking has moved", the new time, the old one, the business's phone and the "Manage your booking" link; `backend/emails/booking-moved-notification.tsx` to the business: "Moved: <service>, <new time>", the old time, the customer's name, phone and email, Reply to the customer. Never the product's name.
 - Sent after the move, not awaited: Jane's when she gave an email, the business's always. Keys `booking-moved/<id>/<sequence>` and `booking-moved-notification/<id>/<sequence>`; each that went gets an `email_sent` entry.
 - `npm run email:preview` writes the two new emails as well.
 - **Done when** saved tests, with Resend faked: a move sends both, Jane's invite carrying the booking's UID, the new times and the new sequence; a second move sends a higher sequence under new keys; a cancel after a move carries a sequence above the move's; a phone-only booking sends only the business's; a repeat of the same move sends nothing more; Resend failing keeps the move and logs one line; the templates show the time in the business's zone, no product name, the customer's text as text; the preview opened by hand, screenshots in the log.
 - ✓ **7b.5 Jane's page: change the time.**
 - On `frontend/components/booking-page/booking-page.tsx`, beside Cancel: "Change the time" opens the free times a week at a time (earlier and later weeks, never before today), grouped by day in the business's zone. Who comes first: "Any available" or a named person who offers the service (decision 10), opening on her own person (decision 14), the week reloaded when it changes. Picking a time asks once more ("Move to Tuesday, October 13 at 10:00 a.m. with Mei?", "Keep the current time" focused first), locks while sending, and ends on "Your booking has moved" with the new time read out and focused.
 - States: no free times that week (said, with the next week a press away), a time taken meanwhile ("That time was just taken. Pick another." and the times reloaded), already started (the business's phone), cancelled, and a failure that keeps the choice usable. Through the typed public client, never with the login cookie.
 - **Done when** the frontend build and lint pass; checked by hand in the browser at phone width on a dev booking: the times, a week with none, the confirm step, the moved booking, a time taken meanwhile, a started booking; focus and announcements checked; the screenshots in the log.

Files / areas

- `backend/lib/scheduling/find-free-times.ts` (`ignoreBooking`), and the Google busy time it reads
- `backend/lib/booking/`: `move-booking.ts`, `find-booking-page.ts` (`canMove`), the after-move starts beside `booking-event-writes.ts` and `booking-cancellation-emails.ts`
- `backend/lib/calendar/`: `calendar-provider.ts`, `google-calendar-provider.ts` (`updateEventTime`), a new `move-booking-event.ts`
- `backend/lib/email/`: `booking-ics.ts`, a sender for the two new emails through `find-booking-email-context.ts` and `send-and-record-emails.ts`, `send-cancellation-emails.ts` (the sequence)
- `backend/emails/`: `booking-moved.tsx`, `booking-moved-notification.tsx`, the preview script
- `backend/routes/public-booking-page-routes.ts`
- `packages/shared/db/booking-tables/booking-table.ts` (`sequence`), `packages/shared/crm/activity-types.ts`, migration 0018, the business slug rule (where client setup makes the slug; confirmed in 7b.1)
- `frontend/components/booking-page/`, `frontend/lib/api-client.ts`
- their tests

Data / contracts

Free times for a move (`GET /public/bookings/:token/times`, 200):

```
type BookingMoveTimesType = {
  timezone: string; // the business's IANA zone
  people: { id: string; name: string }[]; // who she can pick, by name
  startTimes: string[]; // ISO 8601 instants in UTC, ascending, unique
  lastDate: string; // YYYY-MM-DD, the last date it takes bookings (7b.5's review, F-1
};
```

`from`, `to` and `person` follow the booking form's query rule (`freeTimesQueryValidationSchema`, 31 dates at most). 400 `bad_request`; 404 `not_found`; 409 `already_started` or `already_cancelled`; 503 `unavailable`. The same shape as the booking form's own times route; an unknown `person` answers 404 `not_found`.

Move (`POST /public/bookings/:token/move`, body `{ startsAt: string; personId: string | null }`): 200 the page's view at the new time; 400 `bad_request` (a start that is not a time, a body that is not JSON); 413 for a body over 1 KB; 404 `not_found`; 409 `time_taken`, `already_started` or `already_cancelled`; 503 `unavailable` when a calendar cannot be read. Safe to repeat.

The page's view gains `canMove: boolean` (confirmed and not started) and `personId: string` (the booked person, so the page tells her own time from another person's; 7b.5's review, F-160).

booking gains `sequence integer not null default 0`.

The timeline entry: `type: "booking_moved"`, payload `{ bookingId, fromStartsAt, toStartsAt, fromPersonId, toPersonId, sequence }` (ISO instants; the people since 7b.4's review, F-154, so a move's emails name that move's person even when sent after a later move), `actorUserId` null (the customer did it).

The moved invite: `invite.ics`, `text/calendar; charset=utf-8; method=REQUEST; METHOD:REQUEST`, `STATUS:CONFIRMED`, the booking's UID, `SEQUENCE:<sequence>`, the new times, `DTSTAMP` from the move's entry. The cancelling invite: `SEQUENCE:<sequence + 1>`.

The move emails: keys `booking-moved/<id>/<sequence>` and `booking-moved-notification/<id>/<sequence>`; `email_sent` kinds `booking_move` and `booking_move_notification`.

Testing

Backend Vitest against the local database, as 7a: businesses of the test's own, Google and Resend faked, nothing real sent. Every case on the feature's Simulate page is a saved test under the same name. The frontend has no test runner: the page is checked by hand in the browser (there is no browser test harness), with screenshots in the build log.

Notes for the AI

- Read the booking through its own business: the token gives the booking id, the booking row its business, every other read uses that business.
- Lock the booking row (`for update`) before deciding, so two moves, or a move and a cancel, at once make one outcome.
- Release the old rows before holding the new ones inside the transaction: the overlap rule would refuse a new span that overlaps the booking's own old one otherwise.
- The free-times check runs outside the transaction, as `bookTime` 's does; the overlap rule inside it is the race guard.
- The Google event's id is the booking id without dashes, plus `s` and the move's number for a write after a move (`calendarEventIdOf(id, sequence)`); the saved `calendarEventId` is what updates and removals use.
- From step 7b.3's review, for feature 8 (F-149, F-150): a follow that cannot remove the first person's event, or a cancel that beats the follow, leaves that event behind with nothing pointing at it, and two moves within one follow's Google calls can leave an orphan. The move's retry job must carry the first person and the event id it is removing; a single column on the booking would only half-solve it. F-153 (unverified): a PATCH of an event deleted by hand in Google may answer 200.

- From step 7b.4's review, for feature 8 (F-156): six modules repeat the same start-and-settle background tracker (event writes, moves, removals, and the confirmation, move and cancellation emails), and every test file awaits each one by hand. Feature 8's job runner replaces them; nothing shared is built before it.
- A confirmation retried by feature 8 after a move would carry the new times under the confirmation's old key, which Resend refuses: feature 8 decides (a note, not this feature's work).
- Never put a token, a customer's details or a key in a log line.
- From step 7b.1's review: a business that switches a service off leaves the booking's page open while its move times answer 404 (F-138); Frank decided (decision 13) the booking keeps working, so 7b.2 reads its own service even when switched off. An owner can still change their address through Better Auth's own organization update (F-137), which the reserved slug does not cover; Frank left it for Settings, feature 12 (2026-10-03).
- `bookTime` orders "any available" by the day's booking counts; a move's counts must leave out the booking being moved (7b.2).
- From step 7b.2's review, for 7b.3 (F-141): a move to another person keeps `calendarEventId` while `personId` changes, and nothing records whose calendar holds the event, so 7a's cancel would remove it from the wrong person. 7b.3 moves the event between calendars and must know the old person (the move can hand it over, or the event's owner can be saved).
- From step 7b.2's review (F-145): the move's free check copies about 70 lines of `bookTime` 's; the name lookup is shared now (`find-resource-names.ts`), the rest is carried for when the owner's move (features 11 and 12b) gives a third caller.

Open questions

1. **Who does the moved booking?** Answered by Frank, 2026-10-03: changing the time is booking again, with the same choices as her first booking (decision 10). 2. **How is the booking's own Google event left out?** Answered by Frank, 2026-10-03: crossed off Google's busy answer (decision 11). 3. **Until when can Jane move?** Answered by Frank, 2026-10-03: until the appointment starts (decision 12).

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `db/booking-tables/booking-table.ts` : the booking gains `sequence` , 0 when made and one more for each move, the number every invite carries (decision 6). `crm/activity-types.ts` :

`booking_moved` joins the timeline's types. Migration `0018_booking_moved_and_sequence.sql`, generated, adds the column and widens the type check.

- `zod-validation/.../move-booking-validation-schema.ts`: `{ startsAt, personId }`, the start an ISO instant with `Z` or an offset, the person the usual id rule, or null or left out for any available. The booking itself only ever comes from the signed link.
- `business-name-validation-schema.ts`: a name whose address would be `bookings` is refused ("That name is taken by the app. Add a word to it."), because `/public/bookings/...` is where the customer's page asks. Only when a business is made; a rename through Better Auth's own organization update is left for Settings (F-137).
- `helpers/tel-href.ts` (final review, F-164): the one phone link, with its own test. The panel had written a third copy that lost a backslash, so the "call" link at the last bookable week dialled nothing; the page, the panel and all six emails now import this one, and the backend's and the page's copies are gone.

backend: free times for a move

- `lib/scheduling/find-free-times.ts`: `ignoreBooking ({ id, personId, startsAt, endsAt })` leaves that booking's held rows out of every person's and room's busy time, and crosses its own start to end off the Google busy blocks of its current person only (decision 11). With it, the booking's service is read even when switched off (decision 13), passed down to `find-service-resources.ts` as `serviceMayBeOff`; new bookings still need it on.
- `cross-off-span.ts`: pure and half-open; a block reaching past the booking keeps the parts outside it. Google's free/busy does not say which event is which, so another event the person put on top of the booking is hidden for that overlap, the slip decision 11 accepts.
- `lib/booking/find-booking-move-times.ts`: reads the booking's own business from its row, answers cancelled and started before reading any times, and adds `lastDate`, today in the business's zone plus its horizon, the same date `findFreeTimes` stops at, so the page stops offering later weeks there (7b.5's review, F-159).
- `GET /public/bookings/:token/times`: the booking form's own query rule and answer shape plus `lastDate`; the same 404 for a bad link and for a person who does not offer the service; 409 for started or cancelled; 503 when a calendar cannot be read; `no-store`.

backend: the move

- `lib/booking/move-booking.ts`: the start is checked outside the transaction, as `bookTime` does, for the picked person or for everyone who offers the service, with `ignoreBooking`. A picked person's unreadable calendar answers `unavailable`; with any available, nobody free and a calendar unread also answers `unavailable`, never `time_taken`. Rooms are checked without the booking's own rows, and any available's order counts the day's bookings without this one.
- Then one transaction with the booking row locked first, so two moves, or a move and a cancel, make one outcome. It checks status, start and "already there" again, releases the active held rows before

holding the new span (the overlap rule would refuse a span overlapping its own old one otherwise), saves the times, person, room and `sequence + 1`, and writes one `booking_moved` entry with no actor. When every choice was taken meanwhile it throws, so the release is undone with the rest.

- The same start with the same person, or with any available, answers the booking unchanged and writes nothing (decision 4). The Google follow and the emails start only when this call changed something, so a second press does neither again.
- The entry's payload carries the people as well as the times (`fromPersonId` , `toPersonId`), so a move's emails can name that move's own person even when sent after a later move (7b.4's review, F-154).
- `lib/scheduling/find-resource-names.ts` : `bookTime` 's name lookup moved out and shared by booking, the move and the move's emails. The rest of the move's check is still a copy of `bookTime` 's (F-145).
- `lib/booking/find-booking-page.ts` : the page's view gains `canMove` (the same as `canCancel` , a switched-off service included) and `personId` , so the page tells her own time from another person's by id rather than by name (7b.5's review, F-160).
- `POST /public/bookings/:token/move` : 1 KB at most (413); a body that is not JSON gets the same refusal shape as the rest through the routes' `onError` ; 400 `bad_request` , 404, 409 `time_taken` , `already_started` or `already_cancelled` (a new refusal code), 503 `unavailable` ; 200 is the page's view at the new time.

backend: Google

- `lib/calendar/calendar-event-id-of.ts` : takes the move's number. Sequence 0 keeps the plain id, so a cancel can still find an event whose id was not saved yet; a write after a move gets `<id>s<n>` , because Google refuses an id it once deleted in a calendar, and moving back to the first person must never be refused.
- `google-calendar-provider.ts` gains `updateEventTime` , a `PATCH` of the event's start and end; 404 and 410 answer false.
- `move-booking-event.ts` : with the same person, the event is moved in place (decision 7), and one found but never saved has its id saved; one that is not there is written afresh. With another person, the new person's event is written first under a fresh id, then the first person's removed in its own try, so a first calendar that needs reconnecting never stops the new person getting it (7b.3's review, F-148). The move hands over who held the booking before, because the booking keeps no record of whose calendar holds the event. A booking cancelled meanwhile gets nothing; the cancel's removal runs.
- `write-booking-event.ts` writes with the booking's sequence; `remove-booking-event.ts` removes by the saved id first, so a cancel after a person change removes the event from the right calendar.
- `lib/booking/booking-event-moves.ts` : started after the transaction, never awaited (decision 5), in the shape of the writes and removals; a test can await them.

backend: the emails

- `lib/email/booking-ics.ts` : takes `sequence` . The confirmation sends 0 (now tested, F-155), each move its own number, and the cancel `sequence + 1` , so a calendar always takes the latest invite (decision 6).
- `emails/booking-moved.tsx` (Jane: "Your booking has moved", the new time, where it moved from, who and where, "Manage your booking" and the business's phone, the updated invite attached) and `booking-moved-notification.tsx` (the business: "Moved: <service>, <new time>", the old time free again, the customer's name, phone, email and address, Reply to the customer). The business is always told; Jane only when she gave an email.
- `lib/email/send-move-emails.ts` : reads the move from its own timeline entry, found by booking and sequence: its times, its person, and its moment as the invite's stamp, so a retry sends the very same email under keys `booking-moved/<id>/<sequence>` and `booking-moved-notification/<id>/<sequence>` . A booking cancelled since sends nothing; the cancellation says so. Through the shared reading and sending files 7a made, with `email_sent` kinds `booking_move` and `booking_move_notification` .
- `lib/booking/booking-move-emails.ts` starts them without waiting, as the cancellation's do. `npm run email:preview` writes the two new emails.

frontend

- `lib/api-client.ts` : `fetchBookingMoveTimes` and `moveBookingPage` , through the typed public client, never with the login cookie; each answer becomes one plain state.
- `components/booking-page/change-time-panel.tsx` : Who first, then the free times a week at a time from today, grouped by day in the business's zone. Who opens on her own person, any available or anyone else who offers the service one pick away, and falls back to any available only when the times route refuses her person (decision 14, from the final review, F-168). Her current time is hidden unless another person is picked, since moving there would change nothing.
- Picking a time asks "Move to <time> with <person>?" (or "with any available person"), "Keep the current time" focused first; "Yes, move it" locks while sending. A time taken meanwhile reloads the times with "That time was just taken. Pick another."; any other failure keeps the question and puts focus back on "Yes, move it" (F-157).
- Earlier is off on the first week; Later and "Show the next week" stop at the week holding `lastDate` , which says how far ahead the business books and offers its phone. One live region, always mounted, reads out each week and its count (F-162); dates are built from `formatToParts` (F-163); the week label is short enough for 320px (F-161). A late answer for another week or person never replaces the current one. Its `dateIn` and `addDays` are untested copies of the backend's (F-172).
- `components/booking-page/booking-page.tsx` : "Change the time" in the business's colour above Cancel when `canMove` ; the panel takes the buttons' place; a move ends on "Your booking has moved" with "It's now <time>.", focused; a booking that can no longer move goes back to the page's started or

cancelled state. Closing the panel returns focus to its button, and a failed cancel now returns focus to "Yes, cancel it", the same gap.

Packages

None installed.

Tests

Every case on the feature's Simulate page is a saved test under the same name. Google and Resend are faked in every test that could reach them. The race of a time taken between the check and the hold has its own file, `move-booking-taken-meanwhile.test.ts`, which shows the check an empty world while the database holds the time, so the hold fails inside the transaction and everything is undone (7b.2's review, F-140). The final review added three tests with an unreadable calendar (F-167): a picked person whose connection needs reconnecting answers 503 and nothing changes, any available with every calendar unreadable answers 503 rather than `time_taken`, and the move-times route answers 503 when Google fails. The customer's page has no saved test, because the frontend has no test runner yet; it was checked by hand in the browser at phone width, with screenshots in the build log. Final count: 573 backend and 112 shared tests.

Review history

Every step was reviewed by a fresh reviewer before the next began; no P0 or P1 was ever open. The step reviews raised F-135 to F-163, each fixed before the next step or carried with Frank's agreement; F-138 became decision 13. The final review passed over the whole feature four times, raised F-164 to F-172, and closed its three P2s once fixed: the dead call link (F-164), the unreadable calendar during a move (F-167), and the default that could hand Jane to another person without saying so (F-168), which became decision 14. It passed at `851fadb`.

Carried forward

- F-149 (P2), F-150 and F-169 (P3): a follow that gives up on the first person's calendar, two moves inside one follow's Google calls, or a failed write into the new person's calendar can leave an event behind that nothing points at. Feature 8's retry job must carry the first person and the event id it is removing.
- F-153 (unverified): a `PATCH` of an event deleted by hand in Google may answer 200, so the move reports moved while the event stays hidden.
- F-156 (P3): the sixth copy of the start-and-settle tracker; feature 8's job runner replaces all six.
- F-145 (P3): the move's free check copies `bookTime`'s, carried until the owner's move (features 11 and 12b) gives it a third caller.
- F-137 (P3): the reserved slug can still be taken by renaming through Better Auth; Settings, feature 12, which also brings a business's own cutoff for moves.

- F-170 (P3): the move-times privacy test reads a 503 when its file runs in order. F-171 (P3): AGENTS.md's branch example has no build-plan number. F-172 (P3): the panel's date helpers, for `packages/shared` or the dashboard.
- F-146 and F-161 stay marked fixed; a week across two months at 320px still wants one look in the browser.
- Feature 8: a confirmation retried after a move would carry the new times under the confirmation's old key, which Resend refuses.
- At deploy: migration 0018.
- Features 11 and 12b: the owner's move can call `moveBooking`, with its own actor and wording.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-32, F-47, F-58, F-62, F-94, F-95, F-116, F-128, F-134, F-137, F-145, F-146, F-149, F-150, F-153, F-156, F-161, F-169, F-170, F-171, F-172 stay in the live ledger.

7b/F-135 [P3] closed - The Google tests for a move depend on running in order and never pin the cross-off to the booking's own person

File: backend/routes/public-booking-move-times-routes.test.ts:206-245 (cross-off: backend/lib/scheduling/find-free-times.ts:135) **Found:** 2026-10-03 by independent review of step 7b.1 (scope: a55c8ee..548c727; lenses: quality, security, performance, tests) **Why it matters:** "the booking's own Google event does not block" (:238) relies on Ana's calendar connection saved inside the test before it. Run alone (`vitest -t "own Google event"`) Ana has no connection, Google is never asked, and the test passes even with the cross-off switched off: reproduced in this review. In the full file it does catch that break. Separately, decision 11 crosses off the booking's span for its current person only; changing the guard at :135 to cross it off for every person (`if (ignoreBooking)`) left all 9 tests green, because only Ana has a connected calendar. Another person's own Google event at the same time as Jane's booking would then be hidden, and nothing would say so. **Suggested fix:** Give the Google cases their own setup (connect Ana in a `beforeAll` of a nested `describe`, or in each test), and add a case where Mei is connected too and Google answers the same 9:00 to 10:00 for both: Ana's 9:00 is offered, Mei's is not. **Resolution:** Fixed 2026-10-03: each Google test saves its own connection (saving again only updates), so each passes or fails alone; a new test, "the booking's own hour is crossed off only for its own person", connects Ana and Mei with the same 9:00 to 10:00 and expects Mei's 9:00 and 9:30 to stay busy. Removing the cross-off fails the own-event test run alone; crossing off for every person fails the new test. Closed 2026-10-03 by independent review of step 7b.2: re-read find-free-times.ts:138 and the test file; the cross-off switched off fails "the booking's own Google event does not block" run alone, and `if (ignoreBooking)` fails "crossed off only for its own person", both reproduced and restored.

7b/F-136 [P3] closed - No test asks a booking's move times for a person who does not offer its service

File: backend/routes/public-booking-move-times-routes.test.ts:247-258 (answer: backend/lib/booking/find-booking-move-times.ts:57) **Found:** 2026-10-03 by independent review of step 7b.1 (scope: a55c8ee..548c727; lenses: quality, security, performance, tests) **Why it matters:** The step's plan promises 404 for a person who does not offer the service, and the contract says an unknown `person` answers 404 `not_found`. That is also the answer that keeps another business's person out (findFreeTimes checks `person` against the service's own people, inside the booking's business). The behaviour is right today, but changing :57 to answer an empty 200 when findFreeTimes gives null left every test green in this review. The tenant check itself is tested only on the booking form's route, not on this one. **Suggested fix:** Add one case: Jane's link with `person=` a resource id of another test business, and one with the clinic's Room 3 id, each answering 404 with the same body as a bad link. **Resolution:** Fixed 2026-10-03: a new test asks for Room 3 (a place, not a person who can be picked) and for an unknown id, both answering the same 404; answering 200 with no times instead fails it. Closed 2026-10-03 by independent review of step 7b.2: re-read find-booking-move-times.ts:57 and the test; answering an empty 200 for a null fails "a person who does not offer the service, or another business's, answers 404", reproduced and restored.

7b/F-138 [P3] closed - A booking whose service was switched off opens its page, but its move times answer "This link does not open a booking"

File: backend/lib/booking/find-booking-move-times.ts:15,57 (page: backend/lib/booking/find-booking-page.ts:86) **Found:** 2026-10-03 by independent review of step 7b.1 (scope: a55c8ee..548c727; lenses: quality, security, performance, tests) **Why it matters:** findFreeTimes returns null for an inactive service, and the times route turns that into the bad-link 404. findBookingPage does not look at `bookingLink.active`, so Jane's page still opens, and 7b.2 plans `canMove` as the same as `canCancel`. When the business switches a service off with bookings still ahead, the page will offer "Change the time" and the times call will tell her the link opens no booking, which is false. No test or spec line covers the case. **Suggested fix:** Decide in 7b.2's plan whether a switched-off service can still be moved. If not, make `canMove` false for it and let the page say to call the business; if so, read the service without the `active` filter for a move. **Resolution:** Decided by Frank, 2026-10-03 (decision 13): switching a service off stops only new bookings; an existing booking keeps Change the time. Fixed by step 7b.2, which reads the booking's own service even when switched off. Built in 7b.2 (1731845): findServiceResources and findFreeTimes read the booking's own service when switched off; tested by "a booking whose service was switched off can still move, while new bookings cannot". Closed 2026-10-04 by independent review of step 7b.3: re-read find-free-times.ts:64-72 and move-booking.ts:82-93 (the booking's own service read with `serviceMayBeOff`) and the test, which asserts the form's times answer 404 while the move's times and the move answer 200.

7b/F-139 [P3] closed - The spec names `ignoreBookingId` and "the day's booking counts"; the code has `ignoreBooking` and no such counts

File: blueprint/context/current-feature.md:117-120,144,234 **Found:** 2026-10-03 by independent review of step 7b.1 (scope: a55c8ee..548c727; lenses: quality, security, performance, tests) **Why it matters:** The built input is `ignoreBooking: { id, personId, startsAt, endsAt }` (find-free-times.ts:39), because the Google cross-off needs the person and the span. The spec's 7b.1 bullet, 7b.2's plan (which will call it) and Files still say `ignoreBookingId`, and 7b.1 promises the booking is left out of "the day's booking counts", which findFreeTimes does not have. AGENTS.md asks for a wrong spec to be corrected before the next step builds on it. **Suggested fix:** Change the three lines to `ignoreBooking` and drop the booking-counts clause. **Resolution:** Fixed 2026-10-03: the spec says ignoreBooking with its four fields and crossOffSpan, drops the daily counts from 7b.1, and notes that 7b.2's any-available order must leave the moved booking out of the day's counts. Closed 2026-10-03 by independent review of step 7b.2: re-read current-feature.md; 7b.1's bullet names `ignoreBooking` with its four fields and `crossOffSpan`, no booking counts remain in 7b.1, and the Notes for the AI carry the 7b.2 counts rule.

7b/F-140 [P2] closed - A hold that fails inside the move's transaction is never tested; committing the release with no new hold leaves every test green

File: backend/lib/booking/move-booking.ts:221 (test: backend/routes/public-booking-move-routes.test.ts:254) **Found:** 2026-10-03 by independent review of step 7b.2 (scope: 7059d79..1731845; lenses: quality, security, performance, tests) **Why it matters:** The step's central promise is "a time taken meanwhile leaves her booking as it was" (decision 10): inside the transaction the old rows are released first, so only the thrown `EveryChoiceTakenError` rolls that release back when the new hold fails. The test named "a time taken meanwhile" books 13:00 before the move starts, so the free-times check outside the transaction refuses it and the transaction never runs. Replacing :221 with `return { moved: false, reason: "time_taken" } as const` (which commits the release and leaves a confirmed booking holding no time, so its slot can be booked twice) left all 529 backend tests green in this review. The code is right today; nothing guards it. **Suggested fix:** Add a test in the shape of book-time-taken-meanwhile.test.ts: mock `findCommitments` (or `findFreeTimes`) so the check sees the new time free while the database already holds it for another booking, then assert 409 `time_taken`, the booking's times, person and sequence unchanged, its old row still `active`, and no `booking_moved` entry. **Resolution:** Fixed 2026-10-03: move-booking-taken-meanwhile.test.ts shows the check an empty world while the database holds the new time, so the hold fails inside the transaction; the booking, its released-then-restored row and the timeline stay untouched. Committing the release instead fails it. Closed 2026-10-04 by independent review of step 7b.3: replacing move-booking.ts:228's throw with a `time_taken` return failed "a hold that fails inside the move undoes all of it"; restored.

7b/F-141 [P2] closed - A move to another person leaves its Google event in the old person's calendar, and the booking keeps no record of whose calendar that is

File: backend/lib/booking/move-booking.ts:226 (cancel's removal: backend/lib/calendar/remove-booking-event.ts:21,28) **Found:** 2026-10-03 by independent review of step 7b.2 (scope: 7059d79..1731845; lenses: quality, security, performance, tests) **Why it matters:** The move overwrites `personId` and keeps `calendarEventId`, which points at an event in the old person's Google. 7a's cancel removes the event from `row.personId`'s calendar, now the new person's: Google answers 404, google-calendar-provider.ts:97 treats that as gone, `calendarEventId` is cleared, and the old person's event stays at the old time with no log line, still blocking that person's free times. On this branch, a person-changing move followed by a cancel does exactly that. 7b.3 plans to remove the old person's event after the move, but it runs after the transaction, not awaited, and may fail (decision 5); once it has, the old person exists nowhere: not on the booking, and not in the `booking_moved` payload (`{ bookingId, fromStartsAt, toStartsAt, sequence }`). Feature 8's retry and 7a's cancel then cannot find the event. **Suggested fix:** Settle it in 7b.3's plan, before building on it: keep whose calendar holds the event (a `calendarPersonId` beside `calendarEventId`, set when the event is written and cleared when it is removed), and have removal and update use it rather than `personId`; or at least add `fromPersonId` and `toPersonId` to the `booking_moved` payload and the spec's contract. **Resolution:** Carried to 7b.3's plan (spec, Notes for the AI): the event moves between calendars there and must know the old person. Fixed by step 7b.3: the move hands over who held the event; a person change removes it from that person's calendar and writes it into the new one under an id carrying the move's number, saved; the cancel removes by the saved id. Tested: "a cancel after a person change removes the event from the new person's calendar". Not closed by independent review of step 7b.3 (2026-10-04): the happy path is fixed (removing the old person's delete fails "a move to another person removes the old person's event and writes the new person's"), but the booking still keeps no record of whose calendar holds the event, so a follow that fails, or that a cancel overtakes, leaves it in the old person's calendar where nothing can find it. That half carries on as F-149. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): move-booking-event.ts:80-90 writes the event into the new person's calendar and removes it from the first person's by the saved id, and remove-booking-event.ts:140-143 removes by the saved id; "a cancel after a person change removes the event from the new person's calendar" covers it. The unrepaired half (no record of the event when the follow gives up or a cancel overtakes it) is owned by F-149, still open, so it is not tracked twice.

7b/F-142 [P3] closed - No move test uses a room or moves with any available, so the room's own-row filter and the day's counts can be removed with every test green

File: backend/lib/booking/move-booking.ts:147,169 (tests: backend/routes/public-booking-move-routes.test.ts) **Found:** 2026-10-03 by independent review of step 7b.2 (scope: 7059d79..1731845; lenses: quality, security, performance, tests) **Why it matters:** The test clinic has no rooms, and the only any-available press is one already at its time. Removing `row.bookingId !== bookingId` from the room check (:147), which would refuse a Face and Body move 30 minutes later in the same room though the times route offers it, left the 12 move tests green; so did removing the filter that keeps the moved booking out of the day's counts (:169), which the spec's Notes for the AI call out for this step. Decision 10's any-available move (the order, the person it lands on) is untested end to end. **Suggested fix:** Give one test

clinic a room ticked for the service and move Jane 30 minutes later in it; add an any-available move to a new time where the counts decide the person (Ana with Jane's booking and one other that day, Mei with one), asserting Jane stays with Ana. **Resolution:** Fixed 2026-10-03: tests for a move in a room (Room 3 held again at the new time), any available going to whoever is free, and any available not counting the booking being moved; counting the room's own rows or the day's own booking each fails one. Closed 2026-10-04 by independent review of step 7b.3: dropping the booking filter at move-booking.ts:150 failed "a move in a room takes the room again at the new time", and at :172 failed "any available does not count the booking being moved"; restored.

7b/F-143 [P3] closed - The move route lacks the booking form's body limit and its refusal for a body that is not JSON

File: backend/routes/public-booking-page-routes.ts:78-87 (the form's: backend/routes/public-bookings-routes.ts:28-37) **Found:** 2026-10-03 by independent review of step 7b.2 (scope: 7059d79..1731845; lenses: quality, security, performance, tests) **Why it matters:** The booking form's POST caps the body (`bodyLimit`) and turns Hono's malformed-JSON exception into the refusal shape. The move route has neither, and its validator runs before the token is read, so anyone can make it parse any size of body. Probed in this review through the built app: `{not json` answers 400 `text/plain` "Malformed JSON in request body" (not `{ error: { code, message } }`, which the page will read in 7b.5), and a 5 MB body is parsed in full before the 400. Rate limits are feature 9, but the size cap is this repo's own pattern for public POSTs. **Suggested fix:** Mount the same `bodyLimit` and an `onError` that maps a 400 `HTTPException` to `refuse("bad_request", ...)` on `publicBookingPageRoutes`, as `publicBookingsRoutes` does. **Resolution:** Fixed 2026-10-03: the move route has the booking form's `bodyLimit` (1 KB, 413) and its `onError` refusal for a body that is not JSON (400 `bad_request`); tested. Closed 2026-10-04 by independent review of step 7b.3: re-read public-booking-page-routes.ts:38-41,90-93 and the test "a body that is not JSON, or too large, is refused in the same shape" (400 and 413, both `bad_request`, sequence unchanged).

7b/F-144 [P3] closed - The spec's move contract says 400 `invalid_start` and no 503; the route answers 400 `bad_request` and 503 `unavailable`

File: blueprint/context/current-feature.md:280 (route: backend/routes/public-booking-page-routes.ts:84,104) **Found:** 2026-10-03 by independent review of step 7b.2 (scope: 7059d79..1731845; lenses: quality, security, performance, tests) **Why it matters:** Data / contracts lists the move's answers as 200, 400 `invalid_start`, 404, 409. The route answers `bad_request` for a bad body (as every other route does) and also 503 `unavailable` when a calendar cannot be read. 7b.5 builds the page's states from this contract, so a page written to it would miss the 503 and look for a code that never comes. AGENTS.md asks for a wrong spec to be corrected before the next step builds on it. **Suggested fix:** Change the contract line to 400 `bad_request` and add 503 `unavailable`. **Resolution:** Fixed 2026-10-03: the spec's move contract says 400 `bad_request`, 413 and 503 `unavailable`, as the code answers. Closed 2026-10-04 by independent review of step 7b.3: re-read the spec's Move contract (400 `bad_request`, 413, 404, 409, 503 `unavailable`), which matches the route.

7b/F-147 [P3] closed - The activity types test is still named "the nine kinds" while it expects ten

File: packages/shared/crm/activity-types.test.ts:6 **Found:** 2026-10-03 by independent review of step 7b.2 (scope: 7059d79..1731845; lenses: quality, security, performance, tests) **Why it matters:** The assertion moved to 10 for `booking_moved`; the name did not, so a failing run would report the wrong expectation. **Suggested fix:** Rename it "are the ten kinds of timeline entry, each once". **Resolution:** Fixed 2026-10-03: the test says ten kinds. Closed 2026-10-04 by independent review of step 7b.3: activity-types.test.ts:6 reads "are the ten kinds of timeline entry, each once" and asserts 10.

7b/F-148 [P2] closed - A move to another person writes nothing into the new person's calendar when the old person's calendar needs reconnecting or refuses the delete

File: backend/lib/calendar/move-booking-event.ts:79-82 (backend/lib/calendar/get-fresh-access-token.ts:44) **Found:** 2026-10-04 by independent review of step 7b.3 (scope: 3ba8593..c855db2; lenses: quality, security, performance, tests) **Why it matters:** The old person's key and delete come first, and either one throwing ends the whole follow before `forget()` and `writeBookingEvent`: `getFreshAccessToken` throws `CalendarReconnectNeededError` for a connection marked `needs_reconnect`, and `deleteEvent` throws on any answer but 2xx, 404 and 410. Probed in this review with temporary tests (removed after): Jane booked with Ana, both Ana and Mei connected, Ana's connection set to `needs_reconnect`, Jane moved to Mei: no Google call at all, Mei's calendar gets nothing, and `calendarEventId` still names Ana's event. The same with Ana's DELETE answering 503. Mei then works an appointment that is missing from her own calendar because someone else's calendar is broken, and feature 8 cannot redo it (F-149). A Google refresh token that expires or is revoked is enough to put a connection in `needs_reconnect`. **Suggested fix:** Make the two calendars independent: write into the new person's calendar whatever happened in the old one (write first, or catch and log the old side's failure on its own line), and test the old person needing reconnect, the old delete failing, and the old person having no calendar while the new one does. **Resolution:** Fixed 2026-10-04: on a person change the new person's event is written first, then the first person's removed inside a try; a first calendar that needs reconnecting or refuses the removal no longer stops the write, and the failure is logged. Tested with a connection needing reconnection and a removal answering 503. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): move-booking-event.ts:81-89 writes the new person's event before the first person's removal, which sits in its own try; "the new person gets the event even when the first calendar needs reconnecting" and "... refuses the removal" pin it. No new defect found in the reordering.

7b/F-151 [P3] closed - Four of the follow's branches can be broken with every test green

File: backend/lib/calendar/move-booking-event.test.ts (code: backend/lib/calendar/move-booking-event.ts:46,66-71,80; backend/lib/booking/move-booking.ts:208) **Found:** 2026-10-04 by independent review of step 7b.3 (scope: 3ba8593..c855db2; lenses: quality, security, performance, tests) **Why it matters:** Each of these left the 10 event-move tests, and the write and remove tests, green in this review

(each restored): dropping the save of the id after a PATCH that found an unsaved event (66-71), which is what lets a later cancel find 7a's "written, id not yet saved" event; returning "nothing" when the old person has no calendar instead of writing into the new person's (80; the only unconnected test has nobody connected); starting the follow on the in-transaction "already there" answer (move-booking.ts:208; the second-press test stops at the pre-check on :75); and removing the cancelled check (46). The person-change failure path is untested too (F-148). **Suggested fix:** Add: a same-person move whose event exists but whose id was not saved (asserting the plain id is saved after the PATCH); old person unconnected, new person connected (one POST, to the new person); two simultaneous presses to the same new time making one PATCH; a follow for a booking cancelled before it reads making no PATCH or POST. **Resolution:** Fixed 2026-10-04: tests for an event found but never saved (its id saved), a person change with no first calendar (the new person written), two moves at once (one PATCH), and a cancelled booking's event not moved; each of the four breakages now fails its test. Closed 2026-10-04 by independent review of step 7b.4: re-broke three branches at HEAD (the cancelled check, the id save after a PATCH, the follow started on the in-transaction "already there") and each failed its own test, restored after; the fourth (old person unconnected) is now structural, the new person is written before the first calendar is read, and "a person change with no first calendar still writes the new person's" covers it.

7b/F-152 [P3] closed - The spec and three comments still describe the step's first plan

File: blueprint/context/current-feature.md:178-180,248-249,322 (backend/lib/calendar/calendar-event-id-of.ts:4; backend/lib/calendar/move-booking-event.ts:46; backend/lib/calendar/remove-booking-event.ts:4-5) **Found:** 2026-10-04 by independent review of step 7b.3 (scope: 3ba8593..c855db2; lenses: quality, security, performance, tests) **Why it matters:** The 7b.3 bullet says the seam gains `updateEvent(accessToken, event)` and that a 404 writes the event inside it; the code has `updateEventTime(accessToken, eventId, time)` answering false, and the caller writes. Files names a new `update-booking-event.ts`; the file is `move-booking-event.ts`. The Notes for the AI still say the event's id is the booking id without dashes. In code, calendar-event-id-of.ts:4 says the first id is "made, never stored", but write-booking-event.ts:85-88 stores it; move-booking-event.ts:46 says the removal handles a cancel meanwhile, which F-149 shows false after a person change; and remove-booking-event.ts's header reflow leaves "Only a cancelled" alone at the end of a short line. AGENTS.md asks for a wrong spec to be corrected before the next step builds on it. **Suggested fix:** Bring the bullet, Files and Notes to `updateEventTime`, `move-booking-event.ts` and the id with the move's number; reword the two comments and reflow the header. **Resolution:** Fixed 2026-10-04: the spec says updateEventTime and move-booking-event.ts and gives the id rule with the move's number; the comments in calendar-event-id-of.ts and move-booking-event.ts corrected. Closed 2026-10-04 by independent review of step 7b.4: the spec's 7b.3 bullet, Files and Notes name updateEventTime, move-booking-event.ts and the id with the move's number; calendar-event-id-of.ts:1-5 and move-booking-event.ts:48 no longer claim what the code does not do. Only the cosmetic reflow of remove-booking-event.ts:4 ("Only a cancelled" alone at a line end) was left, which misleads nobody.

7b/F-154 [P3] closed - An earlier move's emails, built after a later move to another person, name the later person in "With"

File: backend/lib/email/send-move-emails.ts:67 (backend/lib/email/find-booking-email-context.ts:52,128; backend/emails/booking-moved.tsx:84; backend/emails/booking-moved-notification.tsx:76; backend/lib/booking/move-booking.ts:244-249) **Found:** 2026-10-04 by independent review of step 7b.4 (scope: b56d43a..1524a1b; lenses: quality, security, performance, tests) **Why it matters:** The move's emails take its times from its own `booking_moved` entry, but the person from the booking row as it is when the emails are built (`movedFacts = { ...facts, startsAt }, personName` read from `booking.personId`). The entry's payload carries no person, so an earlier move's emails cannot know who it was with. Today the window is only the few milliseconds between move 1's commit and its email context read, which no customer can hit through the page; the test "an earlier move's emails, sent after a later move, still carry that move's times" stays on one person and so never sees it. It becomes real with feature 8: a retried move email names whoever holds the booking at retry time, beside that move's old time, and the body then differs from the first try under the same Resend key, which Resend refuses within its key window. Not blocking now. **Suggested fix:** Put `toPersonId` (and `fromPersonId`) in the `booking_moved` payload and read that person's name for the move's emails, so every fact in a move's email comes from the move itself; or record it as a note for feature 8 beside the confirmation note already in the spec. **Resolution:** Fixed 2026-10-04 on Frank's yes: the `booking_moved` entry saves `fromPersonId` and `toPersonId`, and the move's emails name `toPersonId`'s person; a test moves Marco to Ana then back to Marco and sends move 1's emails late, which name Ana. Reading the person from the booking row, or saving the old person as the new, each fail it. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): `move-booking.ts:248-250` saves `fromPersonId` and `toPersonId` on the entry and `send-move-emails.ts:58-72` names `toPersonId`'s person; "an earlier move's emails, sent after a later move to another person, name that move's person" pins it.

7b/F-155 [P3] closed - The confirmation's invite number 0 is untested, so it can equal the first move's with every test green

File: backend/lib/email/send-booking-emails.ts:66 (tests: backend/lib/email/send-booking-emails.test.ts, backend/lib/email/booking-ics.test.ts:16,35) **Found:** 2026-10-04 by independent review of step 7b.4 (scope: b56d43a..1524a1b; lenses: quality, security, performance, tests) **Why it matters:** Decision 6 rests on the order $0 \text{ (made)} < 1 \text{ (first move)} < \dots < \text{sequence} + 1 \text{ (cancel)}$. Every move and cancel number is asserted, but the confirmation's 0 is only asserted in the pure `bookingIcs` unit test, which passes its own input. Changing `send-booking-emails.ts:66` to `sequence: 1` left all 568 backend tests green in this review (restored after). With that change the first move's invite carries the same SEQUENCE as the confirmation's, and calendars that follow RFC 5546 ignore the move: the customer's calendar keeps the old time. **Suggested fix:** In the confirmation test that already decodes the invite, assert `SEQUENCE:0`; or in `send-move-emails.test.ts`, assert the first move's number is above the confirmation's actually sent invite. **Resolution:** Fixed 2026-10-04: `send-move-emails.test.ts` asserts the confirmation actually sent carries SEQUENCE:0 before the move sends 1; the confirmation set to 1 now fails that test. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): `send-move-emails.test.ts:219` asserts the confirmation actually sent carries SEQUENCE:0 before the first move's SEQUENCE:1.

7b/F-157 [P3] closed - Focus falls to the page body after a failed move

File: frontend/components/booking-page/change-time-panel.tsx:171,185 (move at :125-157) **Found:** 2026-10-04 by independent review of step 7b.5 (scope: 79acbd8..48d743c; lenses: quality, security, performance, tests, accessibility) **Why it matters:** While sending, both "Yes, move it" and "Keep the current time" are disabled, so the focused button loses focus. When the answer is a failure (503, not found, unreachable) they are enabled again but nothing puts focus back. Checked in the browser with the move answer faked as 503: `document.activeElement` was BODY. A keyboard or screen-reader user hears the alert, then has to find the question again from the top of the page; the spec asks for "a failure that keeps the choice usable". 7a's cancel confirm (booking-page.tsx:280,294) has the same shape. **Suggested fix:** After a failed answer, focus the "Yes, move it" button (or the alert's container with `tabIndex -1`) in a `requestAnimationFrame`, as the other transitions already do. **Resolution:** Fixed 2026-10-04: a failed move puts focus back on "Yes, move it", and a failed cancel on "Yes, cancel it" (the same gap on the same page); seen in the browser with the move answered 503. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): change-time-panel.tsx:167 puts focus back on "Yes, move it" after a failed move, and booking-page.tsx puts it on "Yes, cancel it" after a failed cancel (`yesCancelRef`), both in a `requestAnimationFrame`. Read in code only; no browser run in this review.

7b/F-158 [P3] closed - "Earlier" back to the first week drops focus to the page body

File: frontend/components/booking-page/change-time-panel.tsx:250 **Found:** 2026-10-04 by independent review of step 7b.5 (scope: 79acbd8..48d743c; lenses: quality, security, performance, tests, accessibility) **Why it matters:** `disabled={weekOffset === 0}` disables the very button just pressed when it returns to this week, so it loses focus. Checked in the browser: Later, then Earlier, and `document.activeElement` was BODY. Keyboard and screen-reader users lose their place in the week bar. **Suggested fix:** Keep the button focusable and use `aria-disabled` with an early return in the handler, or move focus to "Later >" when the offset reaches 0. **Resolution:** Fixed 2026-10-04: Earlier back to the first week moves focus on to Later; seen in the browser. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): change-time-panel.tsx:282 moves focus on to "Later" when "Earlier" returns to the first week. Read in code only.

7b/F-159 [P3] closed - "Later" and "Show the next week" never end past the business's horizon

File: frontend/components/booking-page/change-time-panel.tsx:258-266,306-315 (clamp in backend/lib/scheduling/find-free-times.ts:91-94) **Found:** 2026-10-04 by independent review of step 7b.5 (scope: 79acbd8..48d743c; lenses: quality, security, performance, tests, accessibility) **Why it matters:** The times route clamps the range to today plus the business's horizon and answers an empty list beyond it. The page has no upper bound, so past the horizon every week says "No free times this week." with "Show the next week" offered again, forever. It reads as a fully booked business and invites endless presses, each one a full free-times read (Google included). **Suggested fix:** Let the times answer carry the

last bookable date (a business date, no customer data), disable "Later" past it and say "No times can be booked after <date>" instead of offering another week. **Resolution:** Fixed 2026-10-04 on Frank's yes: the move's times answer names lastDate (today in the business's zone plus its horizon, the date findFreeTimes stops at); the page switches Later off on that week, drops "Show the next week", says how far ahead the business books with its phone, and moves focus back to Earlier. A route test pins lastDate; one day short fails it. Seen in the browser: Later stops at Nov 29 to Dec 5. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): find-booking-move-times.ts:336-338 answers lastDate from the same horizon findFreeTimes clamps to (find-free-times.ts:90-93), the route test "the answer names the last date the business takes bookings" pins it, and change-time-panel.tsx:230,297-299,342 switch Later and "Show the next week" off on that week. The tel link added beside it is broken, recorded separately as F-164.

7b/F-160 [P3] closed - The booking's own start is hidden by matching its person by name

File: frontend/components/booking-page/change-time-panel.tsx:200-207 **Found:** 2026-10-04 by independent review of step 7b.5 (scope: 79acbd8..48d743c; lenses: quality, security, performance, tests, accessibility) **Why it matters:** Hiding the current start for "Any available" and for the same person is right (decision 4 answers it as unchanged, and showing it would end on "Your booking has moved" with nothing moved). Matching by name is the weak part: two people with the same name (common in a clinic) make the other person's slot at that time disappear, and a person renamed after the page loaded shows the booking's own start as a move, which then reports "moved" with no change. Checked in the browser with faked answers: by name it works for the plain case (hidden for Any available and Marco, shown for Ana). **Suggested fix:** Compare by id: the page view can carry the booked person's id (a business resource id, not the customer's details, so decision 3 holds), or the times route can leave out the booking's own start for its own person and for any available on the server. **Resolution:** Fixed 2026-10-04: the page view carries personId, and the panel compares ids; the page route test pins the new field. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): find-booking-page.ts returns personId and change-time-panel.tsx:219-225 compares ids, not names.

7b/F-162 [P3] closed - Week changes may not be announced: each status is a freshly mounted live region

File: frontend/components/booking-page/change-time-panel.tsx:278,303,319 **Found:** 2026-10-04 by independent review of step 7b.5 (scope: 79acbd8..48d743c; lenses: quality, security, performance, tests, accessibility) **Why it matters:** "Finding free times...", "No free times this week." and "N free times this week." are three different elements, each mounted with its text already inside. Screen readers announce changes to a live region that already exists; a `role="status"` inserted already filled is announced inconsistently. Focus stays on "Later >", and the week label ("Oct 11 to Oct 17") is not in any live region, so after pressing Later a screen-reader user may hear nothing, or a count without the week it belongs to. Not checked with a screen reader in this review. **Suggested fix:** One persistent `role="status"` element that always exists in the panel, whose text changes: "Finding free times for Oct 11 to Oct 17", then "3 free times, Oct 11 to Oct 17" or "No free times, Oct 11 to Oct 17". **Resolution:** Fixed 2026-10-04: one live

region is always mounted and names the week with its count ("Oct 4 to 10: 38 free times."); the visible lines are plain text. Not tried with a screen reader. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): change-time-panel.tsx:312-314 is one sr-only role="status" element, always mounted, whose text names the week and its count; the visible lines no longer carry live roles of their own. Not tried with a screen reader.

7b/F-163 [P3] closed - Calendar dates rely on en-CA formatting as YYYY-MM-DD

File: frontend/components/booking-page/change-time-panel.tsx:22-32,94 **Found:** 2026-10-04 by independent review of step 7b.5 (scope: 79acbd8..48d743c; lenses: quality, security, performance, tests, accessibility) **Why it matters:** `dateIn` assumes `Intl.DateTimeFormat("en-CA").format` writes YYYY-MM-DD. That is locale data (CLDR), not a guarantee, and browsers have briefly shipped en-CA as M/d/yyyy before. If a browser ever does, `today` is not a date, `addDays` calls `toISOString()` on an Invalid Date and throws a RangeError, and the panel crashes on open. The backend's `localDate` uses `formatToParts` for this reason (backend/lib/local-time/clock-as-utc.ts:24). **Suggested fix:** Build YYYY-MM-DD from `formatToParts` (year, month, day), and reuse one formatter per zone instead of a new one per start time. **Resolution:** Fixed 2026-10-04: `dateIn` builds YYYY-MM-DD from `formatToParts`. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8): change-time-panel.tsx:24-33 builds YYYY-MM-DD from `formatToParts`.

7b/F-164 [P2] closed - The "call" link beside the last bookable week strips every digit from the business's phone

File: frontend/components/booking-page/change-time-panel.tsx:392 **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8; lenses: quality, security, performance, tests) **Why it matters:** The link is built with `phone.replace(/[^\d+]/g, "")`. That class means "anything but the letter d or a plus", so every digit is removed: "+1 780 555 0101" becomes `tel:+` and "(780) 555-0101" becomes `tel:`. It is the only way the panel offers to reach the business once Jane is on the week holding the last bookable date (the F-159 repair's "To book later, call <phone>"); the number shows as text, but tapping it on a phone dials nothing. The page already has the right helper, booking-page.tsx:26 `telHref ([^\d+]/g)`, and the emails have their own `tel-href.ts`; the panel wrote a third copy and lost the backslash. The frontend has no test runner, and the step's browser check looked at the week bar, not at the link's href. **Suggested fix:** Use the page's `telHref` (export it from booking-page.tsx, or move it to a small file beside both) instead of the inline replace, and look at the link's href once in the browser. **Resolution:** Fixed 2026-10-04: one `telHref` now lives in packages/shared/helpers/tel-href.ts with its own test (digits and a leading plus kept); the page, the change-time panel and all six emails import it, and the backend and page copies are gone. Seen in the browser at 375px on dev booking 106ce52c, week Nov 29 to Dec 5: the call link reads `tel:4035550100`. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..5db122e): packages/shared/helpers/tel-href.ts:5 keeps `/[^\d+]/g` and its test pins digits and a leading plus; change-time-panel.tsx:393, booking-page.tsx and all six emails import it, and no other phone replace is left in frontend or backend.

7b/F-165 [P3] closed - Finding and step numbers in three new code comments

File: backend/lib/booking/find-booking-move-times.ts:17; backend/lib/booking/find-booking-page.ts:28; frontend/components/booking-page/change-time-panel.tsx:229 **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..7a353a8; lenses: quality, security, performance, tests) **Why it matters:** coding-standards.md (Comments) rules out history in code comments, finding and step numbers named; F-116 is the same slip from feature 6. The 7b.5 review repairs wrote "(7b.5's review, F-159)", "(7b.5's review, F-160)" and "(F-159)". After `/complete` archives this ledger they become `7b/F-159` and `7b/F-160`, so the bare numbers point at nothing a later reader can find, and each comment already says its reason in words. **Suggested fix:** Drop the parenthesised review and finding references and keep the sentences. **Resolution:** Fixed 2026-10-04: the three comments keep their sentences without the step and finding numbers. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..5db122e): find-booking-move-times.ts:16-18, find-booking-page.ts:28 and change-time-panel.tsx:219,230 carry no step or finding numbers. Two new test file headers still do; recorded separately as F-166.

7b/F-166 [P3] closed - Step numbers in the headers of two new test files

File: backend/lib/calendar/move-booking-event.test.ts:1; backend/routes/public-booking-move-routes.test.ts:1 **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..5db122e; lenses: quality, security, performance, tests) **Why it matters:** coding-standards.md (Comments) rules out history in code comments, step numbers named. The F-165 repair ("step numbers out of comments") cleared the three named comments but left "(feature 7b, step 7b.3)" and "(feature 7b, step 7b.2)" in these two headers, the only step numbers left anywhere in backend, frontend or packages/shared. The feature reference alone is the project's usual form. **Suggested fix:** Drop ", step 7b.3" and ", step 7b.2" and keep the sentences. **Resolution:** Fixed 2026-10-04: both headers name the feature only. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..6c1fa5d): move-booking-event.test.ts:1 and public-booking-move-routes.test.ts:1 name the feature only, and `git grep` finds no `7b.N` step number in backend, frontend or packages/shared.

7b/F-167 [P2] closed - No test makes a calendar unreadable during a move, so the move's "unavailable" answers can turn into "free" with every test green

File: backend/lib/booking/move-booking.ts:126-138; backend/routes/public-booking-page-routes.ts:80-83,118 **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..5db122e; lenses: quality, security, performance, tests) **Why it matters:** The move adds its own handling of a calendar that cannot be read: `isFree` turns `CalendarUnavailableError` into "unreadable", a picked person's unreadable calendar answers `unavailable`, and any available answers `unavailable` rather than `time_taken` when no one is free and some calendar was unreadable; both routes map that to 503. coding-standards.md (Error Handling) makes this a rule: a failed calendar check never reports "free". No move or move-times test ever makes a calendar unreadable: the route tests stub `fetch` to throw but connect no calendar, and the Google tests always answer free/busy (move-booking-event.test.ts:66-68);

searching the move tests for 503 or "unavailable" finds nothing. Changing the catch at move-booking.ts:127 to return "free" would let a move land on top of an event the server never saw, and every test would stay green. bookTime has the matching tests (book-time.test.ts:349 and :461); the move, a copy of its check (F-145), does not. **Suggested fix:** Two saved tests with a connected calendar whose free/busy answers 500 (or a refresh that fails): a move to that picked person answers 503 `unavailable` and changes nothing; the move-times route for that person answers 503. Optionally, any available with every calendar unreadable answers 503, not `time_taken`. **Resolution:** Fixed 2026-10-04: three tests with an unreadable calendar. A picked person whose connection needs reconnecting: the move answers 503 `unavailable` and the booking, its held rows and the timeline are unchanged. Any available with every calendar unreadable: 503, not `time_taken`. The move-times route with Google answering 500: 503. Mutations caught: the catch in move-booking.ts returning "free" fails two tests, the any-available answer forced to `time_taken` fails one, the routes answering 200 instead of 503 fails three. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..6c1fa5d), read in code (no product code was changed in this review): public-booking-move-routes.test.ts:330-360 gives Mei, then Ana and Mei, a `needs_reconnect` connection, so getFreshAccessToken throws, findFreeTimes throws CalendarUnavailableError for each picked id, and move-booking.ts:127,133-137 answer `unavailable`; the tests assert 503, the booking's start, person and sequence, its active row and no `booking_moved` entry, so a catch answering "free" or a forced `time_taken` cannot pass them. public-booking-move-times-routes.test.ts:263-270 answers Google's free/busy with 500 for a picked Ana and asserts 503 `unavailable`. No new defect in the repair; the shared clinic's lasting connections it adds to are recorded as F-170.

7b/F-168 [P2] closed - "Any available", the panel's default, can hand Jane's booking to another person while her own is free, and the confirm question never says who

File: frontend/components/booking-page/change-time-panel.tsx:91,183 (backend/lib/booking/move-booking.ts:131,180; backend/lib/scheduling/order-any-available.ts:21-23) **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..6c1fa5d; lenses: quality, security, performance, tests) **Why it matters:** The Who select starts at "Any available" (`personId` null), and a move with any available orders every free person by the day's bookings, then by name, with no preference for the person who holds the booking now. Jane booked Mei by name; she opens "Change the time", keeps the default, picks Monday 2:00 p.m. where Ana and Mei are both free with no other bookings that day, and is asked only "Move to Monday, ... at 2:00 p.m.?". The tie goes by name to Ana, so she ends with Ana though Mei was free; she learns it only from the "With" line after the move. The saved test "any available does not count the booking being moved" shows the same (Jane stays with Ana only because Ana sorts first). Decision 10 gives her the booking form's choices but decides neither the default nor this tie; for Face and Body, where the practitioner matters, a silent change of person on a time-only change is the likely surprise. **Suggested fix:** Ask Frank which he wants, then one of: start the Who select at her current person (still a choice she can change); or let a move with any available try the booking's current person first when free; and in either case name the person in the confirm question ("Move to Monday at 2:00 p.m. with Ana?") when the move may change it. **Resolution:** Fixed 2026-10-04 on Frank's call, recorded as decision 14 in the spec: the

Who choice opens on her own person (any available when her person no longer offers the service), and the confirm names who she will be with. Seen in the browser at 375px on dev booking 106ce52c: Who opens on "Marco (estimator)"; the confirm reads "Move to Monday, October 5 at 9:00 a.m. MDT with Marco (estimator)?", and with Any available "... with any available person?". Nothing was moved. Closed 2026-10-04 by independent review of feature 7b (scope: a55c8ee..851fadb): change-time-panel.tsx:93 starts `personId` at `booking.personId` and :98 seeds the Who list with her own person, so keeping the default asks for her person's times and moves with her person; :188-193 names the person, or "any available person", in the confirm question; :137-140 falls back to any available only when the times route refuses her own person (404), as decision 14 says. The backend's any-available order is unchanged, which decision 14 accepts because it is now an explicit pick. No new defect found in the repair; the fallback was read in code, not run in a browser by this reviewer.

Independent review

Status: passed **Target commit:** 851fadb7a38483f4e32fbc032267bfb296a4e97b **Base commit:** a55c8eebeabc63ef9f9fb7bd233d118a134309a3 **Base ref:** main **Spec hash:** d86ebb8100203e2e177a3e442ac1426a60d0d42195f514bf01a8c88fa0714117 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-04T21:08:46Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-04T21:13:51Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`a55c8eebeabc63ef9f9fb7bd233d118a134309a3..851fadb7a38483f4e32fbc032267bfb296a4e97b` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `npm run build --workspace=backend` : pass
- `npm run test --workspace=@scheduleads-app/shared` : pass (17 files, 112 tests)
- `npm run test --workspace=backend` : pass (58 files, 573 tests, local Postgres 18, seeded scheduleads_dev)
- `npm run build --workspace=frontend` : pass
- `npm run lint --workspace=frontend` : pass

- `npm run format:check`: pass

Evidence

- Preflight: HEAD equals the target, `git merge-base main HEAD` equals the base, the spec's SHA-256 matches, and the only tracked change was this file (plus the owner's untracked personal note, outside the review scope).
- Whole delta read fresh (66 files, 20 commits): move-booking.ts (the free check per candidate with `ignoreBooking`, unreadable versus taken, the room check without the booking's own rows, the day's counts without it, the locked transaction re-checking status, start and "already there", release before hold, rollback through EveryChoiceTakenError, follow and emails started only on a real move), find-booking-move-times.ts and its `lastDate`, find-free-times.ts (own rows left out, crossOffSpan only for the booking's own person, switched-off service kept for a move only), cross-off-span.ts, find-resource-names.ts and the bookTime change, find-booking-page.ts (`canMove`, `personId`), the times and move routes (signed token, identical 404s, 409s, 503, 1 KB body limit, non-JSON refusal shape, no-store), move-booking-event.ts and updateEventTime, calendar-event-id-of.ts, the cancel's removal by saved id, send-move-emails.ts (facts from the move's own entry, keys per sequence), both move templates, booking-ics.ts and the cancel at sequence + 1, migration 0018 and the schema, the reserved slug, telHref in shared, change-time-panel.tsx (including the decision 14 repair), booking-page.tsx, api-client.ts (public client only), the Blueprint skill edits.
- Security: every query after the token read filters on the booking row's own business; person and query ids are schema-checked before any query; no customer details or token in any new answer, log line or timeline payload; the booked person's id on the page view is one the customer can already pick.
- Performance: a move with any available re-reads free times once per person (as bookTime does, F-145 carried); nothing unbounded found on the new routes beyond the rate limits left to feature 9.
- Tests lens: no skipped, focused or placeholder tests in the delta's test files; the move, times, Google follow and email suites cover the spec's Done-when lists.
- Ledger: F-168 closed after re-reading the repaired panel; F-146 and F-161 left `fixed` with notes (test file unchanged; 320px still unmeasured).

Findings

- F-172 [P3] open: the change-time panel keeps its own untested copies of the backend's `localDate` and `addDays`.
- Closed this pass: F-168.
- Still `fixed` (P3, not blocking): F-146, F-161. No P0 or P1 is open or fixed.

Remaining risk

- No browser run in this review (Check not required, no dev server started): the decision 14 fallback to any available, focus, announcements and the 320px week bar (F-161) were read in code only.
- No Verify command, no browser test harness and no frontend test runner exist in this project, so the page's behaviour rests on the builder's manual browser checks.
- Real Google and Resend behaviour is faked in every test; F-153 (a PATCH of a hand-deleted event) remains unverified.
- Carried and still open: F-149, F-150, F-169 (orphan events when a follow fails or two moves overlap), F-156 (six background trackers), F-145 (the move's copy of bookTime's check), F-170 (order-dependent privacy test), F-171 (AGENTS.md branch example).
- [blueprint/ai-voice-proposal.md](#) is an untracked personal note outside the review scope; it is not part of the target.